

Deployment Modes for the Man-in-the-Middle Proxy Solution (AWS-Oriented)

Overview

This document describes multiple **deployment models** for the Man-in-the-Middle (MiTM) proxy solution. The solution comprises an intercepting HTTP/HTTPS proxy along with several supporting web services (APIs) that process and analyze web content. While the architecture is primarily described using **AWS** services (EC2, Lambda, S3, etc.), the patterns are cloud-agnostic and can be adapted to other providers (Azure, GCP) with analogous components. We outline each deployment mode, discuss how the components are arranged, and note the scenarios in which each mode is useful. Short paragraphs and lists are used for clarity, as this document is intended for developers, DevOps engineers, and as input to LLMs for further content generation.

Key Components of the Solution: Before diving into deployment options, it's important to understand the main components of the system:

- **Intercepting Proxy Engine (MiTM Proxy):** A persistent process that runs a full HTTP(S) proxy. It intercepts web traffic and performs transformations. This component cannot easily run on a purely serverless platform because it needs to maintain an open network port and handle continuous HTTP connections. Therefore, it typically runs on **persistent compute** (e.g. an EC2 instance or container).
- **Microservice APIs:** A set of FastAPI-based web services that the proxy leverages for various tasks. These include:
 - *MiTM Proxy API* – the primary REST API endpoint that the proxy (or clients) call for processing requests.
 - *HTML Processing Service* – processes or sanitizes HTML content as needed.
 - *Graph Analysis Service* – builds a graph or structure (for example, a DOM or link graph) from the HTML or other inputs.
 - *Semantic Text Service* – performs text analysis or classification (could use NLP/LLM, e.g. AWS Comprehend or custom ML models).
- *Caching Service* – caches fetched pages and analysis results. This uses a flexible storage backend.
- **Storage Backend (Cache Database):** A storage layer for caching and state. In this architecture, cloud object storage (e.g. **Amazon S3**) often serves as the "database" for persistent data (cached pages, results, etc.). By externalizing state to S3, the application servers remain **stateless**, meaning any instance can serve any request without needing session affinity. (Stateless designs externalize state so any server can handle any request, greatly simplifying scaling and improving reliability ¹.)
- **External Services:** The proxy may call out to external APIs (for example, AWS Comprehend or an LLM service for text analysis) unless an offline/local ML service is deployed. In an *offline mode* (described later), even these dependencies can be brought on-premises.

Having these components in mind, we now describe various deployment models, from an **all-in-one monolith** to **fully serverless microservices**, including hybrid approaches. Each model balances trade-offs in complexity, scalability, and resource usage, and all can be viable depending on use-case (even the all-in-one deployment can be suitable for certain production scenarios, not just testing).

Deployment Models

1. All-in-One Single Instance Deployment

In this model, **everything runs on a single server** (single EC2 instance or single container). It is a monolithic deployment where the MiTM proxy and all the supporting services run together on one machine. Key characteristics:

- **Single Machine, Multiple Processes:** The EC2 instance runs the intercepting proxy process continuously (listening on a port for HTTP(S) traffic). On the same machine, the **MiTM Proxy API** (FastAPI web server) also runs, acting as the main application server. The proxy and the API can communicate locally (e.g. via HTTP on localhost or direct function calls). Since both are on one host, there is no network latency between them.
- **In-Memory Microservices:** Although architecturally we have multiple logical services (HTML service, Graph service, etc.), in this deployment they can be loaded into the same process space as the main API service. Using FastAPI's testing/client mechanism, one service can call another internally without actual HTTP requests ². In practice, the MiTM Proxy API process can import and instantiate the other service apps, handling calls as simple function calls. This means **no inter-service network calls** – everything is resolved in-memory. This setup, initially used for integration testing, proves the flexibility of the design: the code does not need to know whether it's calling a remote REST API or an in-process function, since the interface is the same.
- **Flexible Caching Layer:** The caching service in all-in-one mode can be configured for different storage backends. For a completely self-contained deployment, the cache can be in-memory or on the local filesystem. Alternatively, it can point to an external **S3 bucket** (or other cloud storage) to persist data. The caching system is built on an abstraction (e.g. a custom `memoryfs` library) that allows swapping out the underlying filesystem or storage medium without changing code. This means cached pages and results could be stored:
 - In-memory (volatile, for testing or ephemeral use).
 - On local disk (persist across runs on the same instance).
 - In a compressed file (e.g. a zip archive on disk).
 - In cloud object storage like Amazon S3.
- **External Dependencies:** In all-in-one, because all services are local, the only external requirements are access to the target websites (internet or intranet traffic the proxy intercepts) and any third-party APIs for text analysis. If configured to use cloud NLP services, it will call those. Optionally, one can operate in a **fully self-contained mode** by deploying local ML models (see *Offline Mode* below), eliminating calls to external APIs.
- **Use Cases:** This mode is the simplest to deploy and manage. It's ideal for **development, testing**, or small-scale deployments. Everything is in one place, so there's less operational overhead. Notably, this mode could also be viable in production for low traffic scenarios or on specialized

hardware (e.g. an on-prem appliance or a single VM at a customer site). It offers simplicity, though not the same horizontal scalability as other models. However, it can still scale **vertically** (by using a larger instance) or even modestly **horizontally** (see next point).

- **Scaling Option:** Although all-in-one is monolithic, it can be replicated for higher capacity. For example, one could bake a single AMI or container image with this entire stack and run multiple instances behind a load balancer. Because the application is stateless (especially if using S3 for shared storage), an **Auto Scaling Group** of these identical instances can be created behind an **Application Load Balancer (ALB)**. An AWS **CloudFront** distribution or Route53 DNS can sit in front to route requests to the ALB. With this setup, each instance can serve any request and instances can scale out/in as needed. This provides additional resiliency and throughput while keeping the architecture simple (all instances are clones). The important consideration is that all instances must point to the same external storage (e.g. the same S3 bucket) to share cached data and avoid divergence.

2. Split Proxy and API (Two-Tier Deployment)

This model introduces a logical split between the network proxy component and the web service components by running them on **separate servers** or tiers. In other words, the MiTM proxy runs on one host (or cluster), while the FastAPI-based services run on another host (or separate cluster). Key points:

- **Dedicated Proxy Server(s):** One or more EC2 instances are dedicated to running the intercepting MiTM proxy process. This proxy captures traffic and forwards or delegates processing tasks to the API layer. The proxy tier could be scaled independently if needed (e.g. multiple proxies behind a load balancer, if each can handle distinct clients or traffic segments).
- **Dedicated API Server(s):** Separate EC2 instance(s) run the **MiTM Proxy API** service and its dependent microservices. This could be a single instance running the monolithic all-in-one API (with all sub-services in-memory on that instance), or potentially multiple instances if scaling the API tier behind a load balancer. In essence, the API tier in this model is similar to the all-in-one deployment described above *minus* the actual proxy process.
- **Communication:** The proxy server(s) communicate with the API server(s) over the network (likely via HTTP calls). For example, when the proxy intercepts a page, it might send an HTTP request to an endpoint on the API server (e.g. “process this HTML” or “classify this text”). Because they are separate, there is a slight overhead of network communication, but this split allows each tier to be managed and scaled differently.
- **State Management:** As before, state is kept out of the proxy and API servers by relying on an external store (S3) for cache and any transient data that needs sharing. The proxy servers do not store persistent data locally – they either push it to the API (which then stores in S3 via the cache service) or directly to the cache if designed that way. This ensures that adding more proxy instances or API instances won’t lead to inconsistencies.
- **Use Cases:** This two-tier separation can be useful for **security or segmentation** reasons and for scaling specific parts of the system. For example, if the proxy component is resource-intensive (CPU-bound handling many connections or doing TLS interception), you might allocate a powerful instance (or autoscaled group) for proxies, but keep the analysis API on separate instances possibly tuned for memory/CPU as needed. It also provides a clearer separation of

concerns – network-level proxy vs application-level analysis – which could mirror organizational boundaries (maybe a networking team manages the proxies while an application team manages the services). This model is still relatively simple but a step towards a distributed microservice architecture.

3. All-in-One Serverless (Single Lambda Function)

Moving into serverless options, this model keeps all the supporting services together but runs them in a **single AWS Lambda function** instead of on an EC2 instance. The MiTM proxy itself still needs a server, so this is a **hybrid** deployment: an EC2 instance for the proxy, and a single Lambda function that contains the API and all processing logic. Key details:

- **Lambda for API Layer:** We package the MiTM Proxy API and all its sub-services (HTML processing, graph, semantic, cache logic) into one Lambda function. This is effectively an all-in-one microservice backend that runs on-demand. The FastAPI application (with all routes corresponding to the various services) is initialized within the Lambda's handler. This single function is deployed behind an API Gateway or Lambda URL so that it can be invoked via HTTP.
- **Proxy Integration:** The intercepting proxy (running on EC2) will invoke the Lambda's API when it needs to process content. For example, when the proxy receives a page, instead of calling a local process, it will call the API Gateway endpoint (which triggers the Lambda) to process the page or perform classification. The proxy will then wait for the Lambda's response. This implies the proxy instances need network access to the API Gateway endpoint (which they would, if they have internet access or VPC access as appropriate).
- **Benefits:** By using a single Lambda for all services, we **offload the maintenance** of the API tier to AWS. The API layer becomes highly scalable (each invocation can spin up in parallel to handle requests) and **managed (no servers to run for the API)**. This can be cost-efficient, especially if requests are sporadic or the load is highly variable, because Lambda will scale out as needed and scale to zero when not in use. It also simplifies deployment of the API code – just one artifact to deploy to Lambda.
- **Challenges:** Packing everything in one Lambda means that function may be larger and possibly have longer cold-start times (since it loads multiple components). There are memory limits to consider, as this one function must have enough memory to handle, for instance, analyzing a large page and perhaps caching results. Another consideration is **timeout** – Lambda functions have a max execution time (15 minutes as of AWS limits). If any processing is long-running, the function must complete within that or use strategies to split work. In our scenario, most requests (page processing, text classification calls) should be fairly quick, but it's a factor.
- **State Management:** The Lambda function, being stateless by nature, will still use the external S3-based cache. On each invocation it can read/write to S3 via the cache service. This keeps the Lambda lightweight and ephemeral – it does not hold any persistent state between invocations (except for possibly in-memory caching during a warm invocation, but that is not guaranteed across cold starts).
- **Use Cases:** This mode is useful when you want to minimize server management and take advantage of serverless scaling for the application logic. It might be chosen for **environments where running containers/VMs for the API is undesirable**, or to leverage AWS fully-managed services. It still requires the proxy on a server, but everything else is on-demand. This could be

suitable for moderate loads or bursty traffic where the serverless model shines. It's also a stepping stone towards a fully serverless architecture for the microservices.

4. Microservices on Multiple Lambdas (Fully Serverless Backend)

This model takes the serverless approach further by **splitting each microservice into its own AWS Lambda function** (and possibly dedicated API Gateway endpoints for each). The architecture then has the MiTM proxy on EC2, and **multiple Lambda functions** each handling a specific service (HTML processing, Graph, Semantic, Cache, etc.).

- **Separate Lambda Functions:** Instead of one large Lambda, we deploy several smaller Lambda functions:
 - One for the **MiTM Proxy API** (the main entrypoint that the proxy calls for orchestrating tasks).
 - One for the **HTML Processing Service**.
 - One for the **Graph Analysis Service**.
 - One for the **Semantic Text Service**.
 - One for the **Caching Service** (if the cache interface needs its own compute, though if using S3 directly, the cache might be more of a library than a service).

Each function has a well-defined responsibility. For example, the proxy might first call the HTML service Lambda with some raw HTML, then call the semantic service Lambda for text analysis, etc., or it might primarily call the main API Lambda which in turn triggers the others.

- **Service Coordination:** There are a couple of ways this could be orchestrated:
 - The **MiTM Proxy API Lambda** could act as a aggregator: the proxy calls this one Lambda, and then within that Lambda's code it sequentially invokes the other service Lambdas (via AWS SDK calls or HTTP calls to their endpoints) to perform sub-tasks. This would make the Lambda act like a controller, calling others and combining results.
 - Alternatively, the proxy itself might know to call each service Lambda individually in a certain workflow. This is less likely and less clean, so the former approach (one main Lambda coordinating others) is preferable for encapsulation.
- **Event-Driven Alternatives:** Instead of direct calls, one could also design it event-driven (e.g. the proxy puts a message in an SQS queue for "process page" and a Lambda listening on that queue picks it up, which then calls other Lambdas, etc.). However, for simplicity, direct invocation via API Gateway or AWS SDK is the straightforward approach in this context.
- **Benefits:** This is a **fully microservices architecture** in a serverless environment. Each component can scale independently and only incur cost when used. It offers the most granularity: for example, if HTML processing is CPU-heavy, you can allocate more memory (and CPU) to that Lambda without affecting the others. If one service is rarely used, it will rarely run (cost-saving). It also enforces clear module boundaries.
- **Drawbacks:** The complexity is highest in this model. There are many moving parts and each inter-service call is now a **network call between Lambdas** (possibly via API Gateway). That introduces latency and points of failure. Also, developing and debugging a distributed set of Lambdas can be harder than a single process. Deployment and versioning also become more involved (ensuring all functions are deployed and compatible). Additionally, if the services need to share common code or models, you might end up duplicating some code in multiple Lambdas or managing a shared layer.

- **Use Cases:** This approach is suitable for **large-scale production** where maximum scalability and fine-grained control are required. If different services have very different scaling profiles or you want the freedom to deploy/update them independently, this fits. It's also a natural fit if the organization is committed to a microservices strategy or to serverless infrastructure in general. Since the user mentioned this multi-Lambda setup is what is "running now" in development, it suggests this architecture has been achieved and is being evaluated for performance and cost.

5. Kubernetes Deployment (Containerized Microservices)

Another deployment model is to use **Kubernetes** (e.g. AWS EKS or any K8s cluster) to run the microservices as containerized workloads. This is akin to the multi-Lambda microservices approach, but instead of serverless functions, we use long-running containers orchestrated by Kubernetes.

- **Kubernetes Cluster for APIs:** The microservice components (MiTM Proxy API, HTML service, Graph service, etc.) are each packaged into a Docker container image. These are deployed to a Kubernetes cluster as separate deployments (or pods). For instance, you might have a deployment for the HTML processor service, scaled to N replicas based on load, another for the semantic service, etc. An **Ingress** or internal service mesh could expose the MiTM Proxy API service, which then communicates with the others via internal service URLs.
- **MiTM Proxy Deployment:** There are a couple of ways to handle the proxy engine:
 - **Inside Kubernetes:** One could also containerize the MiTM proxy and run it in the cluster (as a Deployment or DaemonSet). This might be practical if you want the proxy to be near the services and easily scalable via K8s as well. Each proxy instance could be a pod that perhaps sidecars with some agent or just communicates out.
 - **Outside Kubernetes (Separate EC2):** Alternatively, the proxy might remain on a dedicated instance or tier outside the cluster, similar to previous models. It would then connect to the Kubernetes services via the cluster's external API endpoint (through an Ingress or LoadBalancer service exposing the MiTM API service). The conversation suggests the author leans toward keeping the proxy separate from Kubernetes – possibly for simplicity or because the proxy might need to run on specialized instances or have special network setup.
- **Scalability and Orchestration:** Kubernetes offers fine-grained control similar to the multi-Lambda approach, but with the flexibility of containers. You can scale each service up/down based on custom metrics, manage rollouts of new versions, and use service discovery to allow the services to find each other. K8s also allows placement control (for example, if some services require GPUs or other resources, or if the proxy pods should run on certain nodes).
- **Complexity:** Running a Kubernetes cluster is an added operational overhead compared to using AWS Lambda. You need to maintain the cluster or use a managed service like EKS. There's also the complexity of networking between pods, Ingress configuration, and ensuring the proxy (if external) can reach the internal services securely (perhaps via VPC networking or an API gateway bridge).
- **Use Cases:** Kubernetes deployment is suitable for organizations that already use K8s for their infrastructure or want **cloud-agnostic container deployment**. It provides more control and the ability to run other sidecar components (for monitoring, logging, etc.) in the same environment. It's a good option if you foresee needing custom scaling logic or if the solution must be

deployable on-premises in a Kubernetes environment. Essentially, this mode trades off the ease of serverless for the flexibility of containers.

6. Fully Offline / Self-Contained Mode

This is less a separate architecture and more an *augmentation* that can be applied to the above modes for a completely **self-contained deployment**. In scenarios where internet access is restricted or using external cloud services is not allowed, the solution can be deployed with **all dependencies satisfied in the local environment**. Key aspects:

- **Local ML/NLP Services:** Instead of calling AWS Comprehend or an external Large Language Model API for text semantics, deploy a container or service with a local **LLM or classification model**. For example, this could be an open-source language model (running via a library or an API like Ollama or HuggingFace pipelines) that the semantic service calls. Similarly, any other external dependency (like a specific Graph API or database) would be fulfilled locally.
- **Multiple Machine Setup:** A likely configuration for offline mode would involve *multiple machines* each handling part of the system:
 - One machine for the **MiTM Proxy** (or a cluster of them if needed).
 - One machine (or cluster) for the **microservice API layer** (could be EC2 or a Kubernetes cluster as discussed in previous modes).
 - One machine for the **LLM/ML services** (if these are heavy, isolating them on their own server with possibly GPU hardware). For example, a server running an instance of an open-source model that the semantic text service queries.

These machines would be networked together in a private environment (no internet egress needed except perhaps to fetch initial model weights, etc., which could also be done offline).

- **No External Calls:** In fully offline mode, the system does not call out to any cloud services or the public internet (except, of course, the proxy still will fetch whatever target pages it needs to intercept – which might be internal sites in a closed network). The S3 storage can be replaced with an on-prem object storage or file server if needed, or a minio service, etc. DNS and CloudFront aren't relevant here since it's likely all within a closed network, though you might still use an internal load balancer and DNS for the internal services.
- **Use Cases:** This mode is meant for **air-gapped environments, on-prem deployments, or high-security installations** where external connectivity must be minimized. It's basically demonstrating that our architecture can be run as a self-contained product. By swapping out cloud dependencies with local ones (and possibly using offline data for anything needed), the entire pipeline from intercepting traffic to analyzing it can run in isolation. Performance will depend on the hardware provided (for instance, running an LLM locally could be slower unless optimized), but it provides maximum control over data and environment.
- **Combination with Other Models:** Offline mode can be combined with any of the structural models above. For example, you could have an all-in-one instance that also includes a local ML model (all in one truly), or a Kubernetes cluster where one of the services is an LLM-serving service. The idea of offline is orthogonal to whether you split services or not – it's about **where dependencies live**. Here, everything lives on the deployed infrastructure.

Considerations and Use Cases for Each Model

Each deployment model has its pros and cons, and the choice may depend on the scale, use case, and constraints:

- **All-in-One Single Instance:** Best for simplicity and quick setup. Useful in development, testing, or small-scale production where one machine can handle the load. Also viable for demo environments or when deploying as a standalone appliance to a client. The downside is limited horizontal scalability (single point of failure unless you replicate the whole instance).
- **All-in-One with Auto Scaling:** A natural extension when you need to handle more traffic but want to keep architecture simple. By scaling identical instances, you maintain simplicity while gaining throughput and high availability. Good for moderate scale with relatively homogeneous load. Ensure externalized state (e.g. S3) is used heavily here to avoid any synchronization issues.
- **Split Proxy/API Tier:** Good when the proxy component and the analysis component have different resource profiles or need to be managed separately. For example, if you have an environment where proxy nodes reside in a particular network zone (close to users or data sources), and the processing nodes can be in another zone. It adds a small complexity but improves flexibility in scaling and possibly security (you can isolate the proxy in a DMZ and keep the processing inside a protected network).
- **Single Lambda (Monolithic Serverless Backend):** Useful to reduce server management overhead for the backend. A good fit if you anticipate bursts of usage or want automatic scaling without running servers 24/7 for the API. Keep in mind the cold-start and execution time limits – ensure typical tasks complete well within one Lambda invocation. Also watch out for deployment package size limits (all code must fit in a Lambda package or layer).
- **Multiple Lambdas (Microservices):** The most scalable and modular, but also the most complex. Choose this when you need the ability to scale different parts of the system independently or update them independently. Also, if you want fault isolation (one service failing doesn't bring down others, and can even be redeployed independently). It is excellent for large teams (each team can own a service) or very high-scale systems. However, consider using this only when needed – if the overhead outweighs the benefits for your scale, a simpler approach might be sufficient.
- **Kubernetes Microservices:** Fills a similar niche as multiple Lambdas (microservices), but in scenarios where you might need more control or on-prem deployment. Use this if you already have Kubernetes expertise or infrastructure. It's particularly useful if the solution needs to be cloud-portable or if certain components won't work in Lambda (for example, using languages/runtimes not well-supported by Lambda, or needing custom OS-level tweaks). K8s can also handle long-running tasks better than Lambda, since pods are not limited by execution time in the same way.
- **Offline Mode:** This is dictated by requirements around data privacy, security, or lack of internet connectivity. If a client or environment cannot call AWS services or send data to the cloud, this mode is the only choice. It usually will be combined with one of the above deployment patterns for structuring the application. The trade-off is you must manage all services (including things like model updates for the ML component) and ensure sufficient hardware is provisioned for components like an LLM.

In practice, the architecture is flexible enough that one could even mix modes for redundancy or testing. For example, one could run a Kubernetes cluster deployment in parallel with a serverless deployment and compare performance, or have an all-in-one instance as a fallback in case the more complex setup fails. Such hybrid arrangements could provide resilience or easier rollbacks. Generally, though, a single mode will be chosen for a production deployment based on the criteria above.

Cloud Provider Compatibility

Although we described these models with AWS in mind (using terms like EC2, Lambda, S3, CloudFront, EKS, etc.), the core ideas translate to other clouds or even on-premises:

- **On Azure:** The EC2 instances could be Azure VMs or Azure Container Instances, Lambda functions correspond to Azure Functions, S3 could be Azure Blob Storage. Azure also has an AKS (Azure Kubernetes Service) for the K8s model. The architecture would remain largely the same in concept.
- **On GCP:** EC2 would be GCE VMs or GKE nodes, Lambda would be Cloud Functions or Cloud Run (Cloud Run could actually host the API services in a serverless container model), storage could be GCS (Google Cloud Storage). GCP's GKE or Anthos can handle the Kubernetes deployment.
- **Other environments:** The all-in-one model could even run on a bare metal server or a local VM. Kubernetes model could be on any CNCF-compliant cluster. The serverless approach could also be implemented with other FaaS platforms or frameworks (OpenFaaS, etc.) if on-prem. The key requirement is that the MiTM proxy itself needs a host where it can run continuously – that remains constant across environments.

In summary, the deployment models provide a spectrum from a simple single-machine setup to a fully distributed, cloud-native microservice architecture. This flexibility allows the MiTM proxy solution to be adapted to various needs, scaling from small deployments to large, and running on AWS or any other cloud with minimal changes. By understanding these options, we can choose the optimal deployment for a given use case and also easily switch modes as the product evolves (for example, starting all-in-one for development and moving to Lambdas or Kubernetes as the user base grows).

¹ Stateless Architecture: The Key to Building Scalable and Resilient Systems

<https://blog.bytebytego.com/p/stateless-architecture-the-key-to>

² Test Client - TestClient - FastAPI

<https://fastapi.tiangolo.com/reference/testclient/>